Implementación de Tetris mediante manipulación de bits en C++

Valentina Posada Vásquez
Natalia Zapata Zapata

Resumen: En el desafío 1 se implementa una versión del juego tetris y este se realiza a partir de manipulación de bits, punteros y memoria dinámica implementando su codificación en c++. En este informe se describe cuál fue el análisis y diseño que se llevó a cabo para su realización, los módulos que se crearon y las funciones que se hicieron para la ejecución óptima del juego, Además, se nombran las dificultades que se presentaron a lo largo de la codificación, errores y posibles soluciones.

Introducción

El juego tetris es un videojuego que consiste en el movimiento de bloques que van colisionando unos con otros hasta llenar el tablero de las mismas. En este desafío se hizo una versión simplificada de este juego, donde no se usará una interfaz gráfica sino que toda su manipulación será por medio de la consola; el juego tetris se realizó teniendo presente técnicas que permitieran hacer un código eficiente y funcional.

Análisis

El juego consta de varias piezas geométricas que caen desde la parte superior y central de un tablero que se va a imprimir en la consola del programa, la idea de este juego es que las piezas a manera que van cayendo aleatoriamente se vayan colisionando o acumulando entre sí. Cada una de estas figuras tiene posibilidades amplias de rotación por lo que el juego tiene como estrategia ir formando filas con las figuras que están representadas por “>”, en el momento en que la fila ya está completada, será eliminada sin perder el progreso de las anteriores y las que estaban en segunda posición de la tabla pasarán a ser las primeras dando así más espacio en el tablero. En el momento en que el tablero queda totalmente cubierto debido a la mala rotación que el usuario realiza, se

imprimirá en pantalla el “game over” que indica que el juego terminó porque no quedó espacio para adecuar más figuras.

El flujo del programa se basará en turnos en los que el usuario solicita y procesa el movimiento de la pieza actual por medio de teclas habilitadas para las diferentes opciones, las posibilidades de movimientos consisten en mover una posición a la izquierda o a la derecha, bajar una posición o rotar la pieza horizontal o verticalmente lo que le permite encajar con las demás, también, las dimensiones del tablero son limitadas por lo cual, en el momento en que una pieza sobre pase dichas dimensiones, aun estando rotada también generará el “game over”.

Esquema - Algoritmos implementados

Para el correcto funcionamiento y ejecución del juego se siguió una estructura modular donde en los archivos “.cpp” se guarda la implementación de las funciones y en los archivos “.h” se declaran dichas funciones. A continuación se describen las funciones que posteriormente son llamadas en el main.

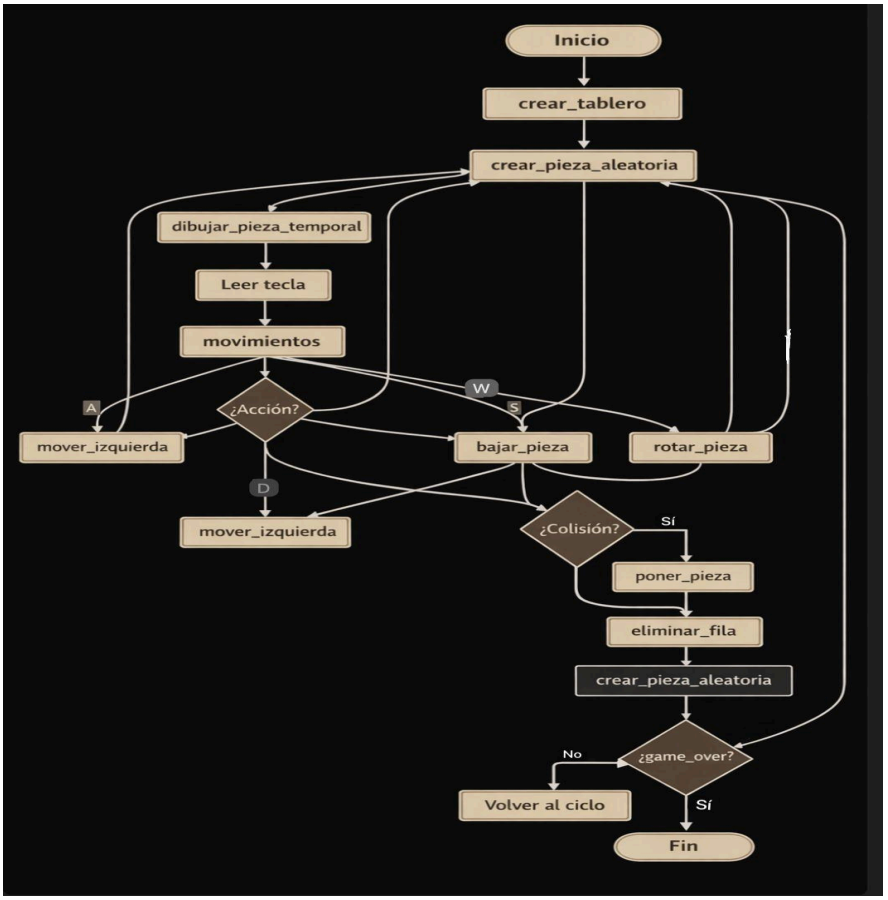

Imagen 1: esquema de algoritmos implementados

1. Tablero

El tablero es la estructura matricial creada a partir de punteros y memoria dinámica y su función es la visualización del juegos la consola, este se conforma por las siguientes funciones:

1.1. Crear_tablero: A su inicio se encarga de validar los datos que ingresa el usuario para que cumpla con los requerimientos básicos, luego, el cuerpo principal de esta función es reservar la memoria dinámica para crear allí los arreglos de punteros que representan las filas y las columnas, los cuales son de tipo unsigned char y sus valores son inicializados en cero. Esta función retorna la estructura creada.

1.2. Dibujar_matriz: Esta función imprime el tablero en consola, para llevar esto a cabo convierte el ancho y el alto ingresado por el usuario en su representación binaria, luego recorre los bits y dependiendo si el bit es 1 imprime “>” para indicar que el espacio ya está ocupado por una pieza o, en su defecto si el bit es 0 imprime “.” Para indicar disponibilidad del espacio.

1.3. Dibujar_pieza_temporal: tiene como objetivo mostrar en consola la pieza pero sin modificar el tablero, es decir, la pieza se superpone temporalmente al tablero pero esta acción no se guarda. Para esto se crea un segmento que va tomando cada fila del tablero y la compara con las máscaras de las piezas, donde las máscaras de las piezas corresponden a las distintas filas que conforman la pieza.

1.4. Poner_pieza: esta función modifica el tablero haciendo que la pieza ya sea parte de él, pero solo se ejecuta si la pieza colisionó con otras piezas, los extremos o el piso del tablero. Para ello se combinan las posiciones de las máscaras con las del tablero.

1.5. Eliminar_fila: tiene como finalidad detectar si hay alguna fila que esté completamente ocupada por piezas, de ser así, esta fila será eliminada. Cuando la fila llena se elimina las demás filas superiores se desplazan hacia abajo generando más filas disponibles.

2. Piezas

2.1. Crear_pieza_aleatoria: es la encargada de generar la pieza aleatoria en el centro del tablero con la que se va a jugar, estas pueden ser cuadrado, línea, T o L. Las máscaras están inicializadas en cero y al escoger la pieza se hacen operaciones de bit para generarlas.

2.2. Game_over: esta es una función que se ejecuta solo cuando hay una colisión de la pieza e indica que el juego debe terminar. Si las piezas colisionan con los extremos del tablero, es decir, se salen de las dimensiones establecidas por el usuario, el juego se da por terminado.

3. Comportamiento de las piezas

3.1. Mover_izquierda: Esta función verifica que la pieza no haya colisionado con otra y que tampoco se encuentre al borde izquierdo para así poder desplazarla una posición hacia la izquierda usando operaciones de bits sobre las máscaras.

3.2. Mover_derecha: Esta función verifica que la pieza no haya colisionado con otra y que tampoco se encuentre al borde derecho para así poder desplazarla una posición hacia la derecha usando operaciones de bits sobre las máscaras.

3.3. Bajar_pieza: Esta función controla el descenso de la pieza en el tablero. Verificando si la pieza puede seguir bajando o si colisiona con el suelo o con otra pieza, en caso de no haber colisión, simplemente baja la pieza una fila. En caso contrario, fija la pieza en el tablero.

3.4. Rotar_pieza: Esta función se encarga de rotar las piezas en el tablero calculando los límites de la pieza para encontrar su centro y luego generar una nueva versión rotada, también, se encarga de validar que las rotaciones no pasen las dimensiones establecidas del tablero.

3.5. Movimientos: Esta función es una de las fundamentales para la ejecución del juego debido a que esta es la encargada de las acciones en el tablero. Dependiendo de la tecla presionada por el jugador (‘a’, ‘d’,

‘s’, ‘w’) llama a la función correspondiente para mover o rotar la pieza. También actualiza el estado de colisión cuando la pieza baja.

Problemas de desarrollo

Durante el proceso de codificar se presentaron algunos problemas principalmente con la manipulación de bytes y al dibujar las piezas. Uno de ellos fue el desbordamiento de bytes al dibujar ciertas piezas en el centro del tablero, para esto se crearon restricciones respecto al byte y al bit que generaba posibles errores.

otro de los problemas más comunes que tiene un código es no tener una validación de datos de entrada, pues el código debe detectar cuando hay una entrada que no corresponde sin necesidad de abortar la ejecución, para ello se realizaron validaciones de entrada para el ancho y la altura del tablero, además de validaciones para las letras de entrada disponibles para la ejecución del juego.

Finalmente, el código presenta un problema con la implementación de una variable que se encarga de detectar la posición de la pieza en el tablero a través de una operación matemática y esa posición al generarse una rotación se altera, por lo que, en algunas ocasiones detecta el piso una o dos posiciones antes de colisionar con el mismo. Todo esto ocurre con la variable posicion_pieza.

Evolución de la solución

La solución propuesta consistió en una organización modular de las tareas, donde cada funcionalidad del juego fue creada de manera independiente. Por ejemplo, se separaron los distintos tipos de movimiento de las piezas (desplazamientos laterales, su descenso y su rotación), permitiendo un mejor control y mantenimiento del código.

Adicionalmente, se implementó una estrategia en la que el tablero y las piezas se manejan de forma independiente durante la ejecución. La unificación entre ambos solo ocurre en el momento en que se detecta una colisión, lo que permite mantener una lógica más clara y evitar modificaciones innecesarias sobre el estado del tablero ya que nos podría provocar un colapso en su ejecución.

Conclusiones

- El implementar el uso de operaciones a nivel de bits para representar las piezas y el tablero resultó ser una estrategia eficiente debido a que reduce el consumo de memoria y mejora el rendimiento del programa
- La modularización del código, separando funcionalidades como movimientos, rotación, generación de piezas y gestión del tablero, facilitó la organización del programa y permitió un desarrollo más claro.
- Este desafío permitió fortalecer conceptos de programación como lo son el uso de punteros, arreglos dinámicos, operaciones bit a bit y control de flujo.